

```

} o1, o2;
/* declare variables o1 and o2 of Circle */

```

Note that you can omit structure tag, but then you cannot declare additional objects of this type elsewhere. For more information, see the “Untagged Structures” below.

Structure is initialised by assigning it a comma-delimited sequence of values within braces, similar to array. For example:

```

/* Referring to declarations from the example above: */
/* Declare and initialize dots p and q: */
struct Dot p = {1., 1.}, q = {3.7, -0.5};
/* Declare and initialize circle o1: */
struct Circle o1 = {1., {0., 0.}}; // radius is 1,
center is at (0, 0)

```

Incomplete declarations

Incomplete declarations are also known as forward declarations. A pointer to a structure type A can legally appear in the declaration of another structure B before A has been declared:

```

struct A; // incomplete
struct B {struct A *pa;};
struct A {struct B *pb;};

```

The first appearance of A is called incomplete because there is no definition for it at that point. An incomplete declaration is allowed here, because the definition of B doesn't need the size of A.

Untagged structures and typedefs

If you omit the structure tag, you get an untagged structure. You can use untagged structures to declare the identifiers in the comma-delimited member-declarator-list to be of the given structure type (or derived from it), but you cannot declare additional objects of this type elsewhere.

It is possible to create a typedef while declaring a structure, with or without a tag:

```

/* With tag: */
typedef struct mystruct { ... } Mystruct;
Mystruct s, *ps, arrs[10]; /* same as struct mystruct
s, etc. */
/* Without tag: */
typedef struct { ... } Mystruct;

```